

Syllabus for Computer Science I: COSC120

Dr. Jordan Hanson - Whittier College Dept. of Physics and Astronomy

Fall 2026

Abstract

Computer Science I introduces computational problem solving and computer programming using Python. The course begins with statements, expressions, variables, primitive data types, input/output, and the relationship between variables and objects. Branching, Boolean logic, iteration, functions, and modules provide the foundation for structured program design. Strings, lists, tuples, and dictionaries are then used to represent and manipulate collections of data. The course continues with object-oriented programming, classes, recursion, inheritance, file input/output, and exception handling. Along the way, students apply Python to numerical computation, searching and sorting, simulation, testing and debugging, and elementary algorithmic analysis. The semester concludes with an introduction to data science using NumPy, pandas, exploratory analysis, and visualization. Programming exercises and a final project emphasize decomposition, abstraction, verification, readable code, and communication of computational results.

Pre-requisites: None. Prior programming experience is not required.

Course credits, Liberal Arts Categorization: 3 Credits, LLB2, QR2

Regular course hours: Mondays and Wednesdays from 8:30–9:50 AM in SLC 416.

Instructor contact information: Discord: 918particle, Email: jhanson2@whittier.edu, Office: SLC 222, Book appointments: <https://calendar.app.google/gYfm8uzRbZ7fW9Dc7>.

Office hours: Use booking link above to schedule office hours.

Attendance/Absence: Students needing to reschedule midterms or other major assessments must notify the professor a few days in advance whenever possible.

Late work policy: Late work is generally not accepted, but exceptions are left to the discretion of the instructor.

Text: Introduction to Python Programming, by Udayan Das, Aubrey Lawson, Chris Mayfield, and Narges Norouzi (OpenStax). The text is available at <https://openstax.org/books/introduction-python-programming/pages/1-introduction>.

Software: Python 3. Students will use the OpenStax code runner and/or a local Python environment. Jupyter notebooks may also be used for numerical, simulation, and data-science exercises.

Grading: The course grade will be a weighted average of assignment scores, and the weights are listed in Tab. 1.

Assignment	Weight	Date
Homework and programming sets	30%	Weekly on Fridays
Midterm 1	25%	Wednesday, September 30th
Midterm 2	25%	Wednesday, November 4th
Final programming project	20%	Finals period, December 9th–12th

Table 1: Major assignments and grade weights. Programming sets include short written exercises, code submissions, testing/debugging activities, and selected textbook problems.

Grade Settings: $< 60\% = F$, $60\% - < 70\% = D$, $70\% - < 80\% = C$, $80\% - < 90\% = B$, $90\% - 100\% = A$. Pluses and minuses: 0–3% of each ten-point interval is a minus, 3–6% is a straight letter grade, and 6–10% is a plus (for example, 79% = C+ and 91% = A-).

Homework and Programming Sets: Due weekly on Fridays. Assignments will combine textbook exercises, short programming problems, debugging, code-reading exercises, and larger applications. Students are expected to submit readable code with meaningful variable names and appropriate comments.

Final Programming Project: Students will design, implement, test, and present a Python program that integrates several course concepts. Projects should demonstrate decomposition with functions and/or classes, use at least one nontrivial data structure, read or generate data, and communicate results clearly.

Student Accessibility Services (SAS) statement: Whittier College is committed to making learning experiences as accessible as possible. If you experience physical or academic barriers due to a disability, you are encouraged to contact Student Accessibility Services (SAS) to discuss options. To learn more about academic accommodations, email sas@whittier.edu, call (562) 907-4825, or visit the SAS office on the ground floor of Wardman Library.

Academic Honesty Policy: <https://www.whittier.edu/academics/academichonesty>

Course Objectives:

- Develop fluency with Python statements, expressions, variables, data types, input/output, and program execution.

- Use Boolean expressions, conditionals, and loops to implement algorithms and solve computational problems.
- Decompose programs into reusable functions and modules, and reason about parameters, return values, scope, and control flow.
- Use strings, lists, tuples, and dictionaries to organize, search, transform, and analyze data.
- Explain elementary algorithmic complexity and implement representative searching and sorting methods.
- Apply numerical programming techniques including floating-point computation, successive approximation, and bisection-style root finding.
- Design programs using classes, objects, encapsulation, inheritance, and methods.
- Apply recursion to elementary computational problems and recognize when iterative or recursive solutions are appropriate.
- Read and write files, work with CSV data, and use exception handling to make programs more robust.
- Use Python as a problem-solving tool for simulation and introductory data analysis, including basic NumPy, pandas, and visualization.
- Test, debug, document, and communicate computational work clearly.

Course Outline:

1. Week 0 - Reading: Chapter 1, Statements.

- (a) First meeting - Wednesday, August 19th, 2026.
 - i. Course introduction and syllabus distribution
 - ii. What is computation? What do computer scientists do?
 - iii. Python programs, statements, and the interpreter
 - iv. Input/output, variables, strings, numbers, comments, and error messages
 - v. *Programming exercise*
 - A. Run first Python programs using the OpenStax code runner and/or local Python
 - B. Input, computation, formatted output, and reading simple error messages

2. Week 1 - Reading: Chapters 2–3, Expressions and Objects.

- (a) First meeting - Monday, August 24th, 2026. *Reading: Chapter 2.*
 - i. Expressions, literals, operators, type conversion, and mixed data types
 - ii. Floating-point representation and floating-point error
 - iii. Integer division, remainder, and the `math` module
 - iv. *Programming exercise, part I*
 - A. Numerical expressions and unit-conversion programs
 - B. Experiments with floating-point precision
- (b) Second meeting - Wednesday, August 26th, 2026. *Reading: Chapter 3.*
 - i. Objects and references; strings revisited
 - ii. Lists and tuples as containers
 - iii. Formatted strings and formatted numerical output
 - iv. Supplemental representation topics: binary, hexadecimal, bits, bytes, and ASCII/Unicode concepts
 - v. *Programming exercise, part II*
 - A. Decimal, binary, and hexadecimal conversion in Python
 - B. String encoding and character-code experiments

3. Week 2 - Reading: Chapters 4–5, Decisions and Loops.

- (a) First meeting - Monday, August 31st, 2026. *Reading: Chapter 4.*
 - i. Boolean values and comparison operators
 - ii. `if`, `elif`, and `else`
 - iii. Boolean operations, precedence, chained decisions, and nested decisions
 - iv. *Programming exercise, part I*
 - A. Classification and decision-making programs
 - B. Testing boundary cases and logical conditions
- (b) Second meeting - Wednesday, September 2nd, 2026. *Reading: Chapter 5.*
 - i. `while` loops and `for` loops
 - ii. Nested loops, `break`, `continue`, and loop control
 - iii. Common iterative program patterns: counters, accumulators, sentinels, and validation
 - iv. *Programming exercise, part II*
 - A. Iterative numerical calculations
 - B. Pattern generation and simple Monte Carlo experiments

4. Week 3 - Reading: Chapters 5–6, Loops and Functions.

- (a) Monday, September 7th, 2026. *Labor Day: no instruction.*
- (b) Meeting - Wednesday, September 9th, 2026. *Reading: Review Chapter 5; Chapter 6.*
 - i. Defining and calling functions
 - ii. Decomposition and abstraction using functions
 - iii. Parameters, arguments, return values, control flow, and variable scope
 - iv. Keyword arguments and reusable numerical functions
 - v. Numerical programming: successive approximation and bisection methods
 - vi. *Programming exercise*
 - A. Refactor an iterative program into small reusable functions and test known inputs/outputs

- B. Implement square-root or root-finding by iteration; compare stopping criteria and floating-point limitations

5. Week 4 - **Reading: Chapters 6–7, Functions and Modules.**

- (a) First meeting - Monday, September 14th, 2026. *Reading: Complete Chapter 6.*
 - i. Function design, interfaces, documentation, and testing
 - ii. Scope, side effects, and debugging function-based programs
 - iii. *Programming exercise, part I*
 - A. Build a small multi-function numerical program
 - B. Develop test cases before final submission
- (b) Second meeting - Wednesday, September 16th, 2026. *Reading: Chapter 7.*
 - i. Module basics and importing names
 - ii. Top-level code and the `if __name__ == "__main__"` pattern
 - iii. Standard-library modules and the `help()` function
 - iv. *Programming exercise, part II*
 - A. Create and import a custom Python module
 - B. Use selected standard-library modules for random numbers and mathematics

6. Week 5 - **Reading: Chapters 8–9, Strings and Lists.**

- (a) First meeting - Monday, September 21st, 2026. *Reading: Chapter 8.*
 - i. String operations, indexing, slicing, searching, and testing
 - ii. String formatting, splitting, and joining
 - iii. *Programming exercise, part I*
 - A. Text parsing and simple data-cleaning problems
 - B. Palindrome and frequency-analysis examples
- (b) Second meeting - Wednesday, September 23rd, 2026. *Reading: Chapter 9.*
 - i. Lists, mutability, iteration, sorting, and reversing
 - ii. Common list operations, nested lists, and list comprehensions
 - iii. Introduction to searching and sorting as algorithms
 - iv. *Programming exercise, part II*
 - A. Implement linear search and compare with Python list operations
 - B. Implement insertion sort or selection sort and test on generated data

7. Week 6 - **Reading: Review Chapters 1–9.**

- (a) First meeting - Monday, September 28th, 2026.
 - i. Review: statements, expressions, objects, decisions, loops, functions, modules, strings, and lists
 - ii. Strategies for testing and debugging
 - iii. Review of representative numerical and algorithmic problems
 - iv. *Programming workshop*
 - A. Catch-up, debugging, and midterm preparation
- (b) Second meeting - Wednesday, September 30th, 2026.
 - i. **First midterm exam**
 - A. Covers Chapters 1–9 and associated programming exercises
 - B. Emphasis on program tracing, control flow, functions, strings/lists, and basic numerical programming

8. Week 7 - **Reading: Chapter 10, Dictionaries.**

- (a) First meeting - Monday, October 5th, 2026. *Reading: Sections 10.1–10.3.*
 - i. Dictionary basics and key-value data
 - ii. Dictionary creation and common operations
 - iii. Choosing lists, tuples, or dictionaries for a problem
 - iv. *Programming exercise, part I*
 - A. Frequency tables and lookup problems
 - B. Build a dictionary from structured input data
- (b) Second meeting - Wednesday, October 7th, 2026. *Reading: Sections 10.4–10.5.*

- i. Conditionals and loops with dictionaries
- ii. Nested dictionaries and dictionary comprehensions
- iii. Elementary algorithmic complexity: constant, logarithmic, linear, and quadratic growth
- iv. Linear search and binary search; why sorted data changes the problem
- v. *Programming exercise, part II*
 - A. Compare linear search and binary search experimentally
 - B. Measure operation counts as input size increases

9. Week 8 - **Reading: Chapter 11, Classes.**

- (a) Monday, October 12th, 2026. *Indigenous Peoples' Day: no instruction.*
- (b) Meeting - Wednesday, October 14th, 2026. *Reading: Sections 11.1–11.5.*
 - i. Object-oriented programming and abstract data types
 - ii. Classes, instances, attributes, and instance methods
 - iii. Abstraction, encapsulation, operator overloading, and class design
 - iv. Organizing classes in modules; interfaces and code reuse
 - v. *Programming exercise*
 - A. Design a simple class to model a real-world entity; create and test multiple instances
 - B. Extend the class with useful operators and validation; begin planning the final programming project

10. Week 9 - **Reading: Chapter 12, Recursion.**

- (a) First meeting - Monday, October 19th, 2026. *Reading: Sections 12.1–12.3.*
 - i. What is recursion? Base cases and recursive cases
 - ii. Recursive factorial, palindrome, and Fibonacci examples
 - iii. Recursion with strings and lists
 - iv. *Programming exercise, part I*
 - A. Trace recursive function calls by hand and in Python
 - B. Compare recursive and iterative solutions
- (b) Second meeting - Wednesday, October 21st, 2026. *Reading: Sections 12.4–12.5.*
 - i. Recursive problem solving and divide-and-conquer thinking
 - ii. Memoization and repeated subproblems
 - iii. Merge sort as a recursive divide-and-conquer example
 - iv. *Programming exercise, part II*
 - A. Implement memoized Fibonacci or a similar recursive computation
 - B. Implement or trace merge sort and compare with quadratic sorting methods

11. Week 10 - **Reading: Chapter 13, Inheritance.**

- (a) First meeting - Monday, October 26th, 2026. *Reading: Sections 13.1–13.3.*
 - i. Inheritance basics and “is-a” relationships
 - ii. Attribute access and inherited methods
 - iii. Method overriding and extension
 - iv. *Programming exercise, part I*
 - A. Build a base class and one or more derived classes
 - B. Test inherited and overridden behavior
- (b) Second meeting - Wednesday, October 28th, 2026. *Reading: Sections 13.4–13.5; begin Chapter 14.*
 - i. Hierarchical inheritance and multiple inheritance
 - ii. Mixin classes and design tradeoffs
 - iii. Introduction to persistent data and files
 - iv. *Programming exercise, part II*
 - A. Extend the Week 8 class design using inheritance
 - B. Save simple program output to a text file

12. Week 11 - **Reading: Chapter 14, Files and Exceptions; review Chapters 8–13.**

- (a) First meeting - Monday, November 2nd, 2026. *Reading: Sections 14.1–14.3.*
 - i. Reading from files and writing to files
 - ii. Paths and files in different locations

- iii. CSV files and structured tabular data
 - iv. *Programming exercise*
 - A. Read, process, and summarize a CSV data file
 - B. Write processed results to a new file
- (b) Second meeting - Wednesday, November 4th, 2026.
- i. **Second midterm exam**
 - A. Covers Chapters 8–13 and associated programming exercises
 - B. Emphasis on strings, lists, dictionaries, classes, recursion, inheritance, and elementary algorithms
13. Week 12 - **Reading: Complete Chapter 14; applied computation.**
- (a) First meeting - Monday, November 9th, 2026. *Reading: Sections 14.4–14.5.*
- i. Exceptions and exception handling
 - ii. **try**, **except**, **else**, and **finally**
 - iii. Raising exceptions and validating program state
 - iv. *Programming exercise, part I*
 - A. Add robust input and file handling to an earlier program
 - B. Design useful error messages and failure tests
- (b) Second meeting - Wednesday, November 11th, 2026.
- i. Computation as a problem-solving tool
 - ii. Computer models of real-world phenomena
 - iii. Random processes, random walks, and simulation
 - iv. *Programming exercise, part II*
 - A. Random-walk simulation or vacuum-cleaner robot simulation
 - B. Collect simulation results for later visualization
14. Week 13 - **Reading: Chapter 15, Data Science.**
- (a) First meeting - Monday, November 16th, 2026. *Reading: Sections 15.1–15.3.*
- i. Introduction to the data-science life cycle
 - ii. NumPy arrays and numerical data
 - iii. pandas data structures and tabular data
 - iv. *Programming exercise, part I*
 - A. Load simulation or CSV data into NumPy/pandas
 - B. Compute descriptive summaries and selected transformations
- (b) Second meeting - Wednesday, November 18th, 2026. *Reading: Sections 15.4–15.5.*
- i. Exploratory data analysis
 - ii. Data filtering, indexing, and visualization
 - iii. Presenting computational results graphically
 - iv. *Programming exercise, part II*
 - A. Visualize simulation or tabular data
 - B. Interpret graphs and identify limitations of the analysis
 - C. Final project development and proposal check
15. Week 14 - **Fall/Thanksgiving Break:** November 23rd–27th, 2026. No instruction.
16. Week 15 - **Reading: Review Chapter 15 and cumulative course concepts.**
- (a) First meeting - Monday, November 30th, 2026.
- i. Integrating functions, data structures, classes, files, and visualization
 - ii. Final-project testing, debugging, and code review
 - iii. Where to go from here: Computer Science II, data structures and algorithms, scientific computing, and data science
 - iv. *Final project workshop*
- (b) Second meeting - Wednesday, December 2nd, 2026.
- i. Course synthesis and cumulative review
 - ii. Student project demonstrations and peer feedback
 - iii. Professional presentation of computational results

iv. *Final project workshop and presentation preparation*

17. Week 16 - **Final instructional meeting.**

- (a) Meeting - Monday, December 7th, 2026.
 - i. Final project demonstrations and peer feedback
 - ii. Final project testing, presentation preparation, and cumulative course wrap-up

18. Final Examination Period - **December 9th–12th, 2026.**

- (a) Final programming-project presentations/submissions will occur during the College-assigned final examination period.
- (b) The exact day and time will follow the Registrar's final-examination schedule for this course section.